

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM 4522 VİZE PROJE RAPORU

Github: github.com/erdemozturkk/BLM4522-Projects

Mustafa Erdem ÖZTÜRK

21290480

Enver Bağcı

Nisan, 2026

İÇİNDEKİLER

İÇİNDEKİLER.....	1
GİRİŞ.....	1
Kullanılan Teknoloji ve Veri Seti.....	1
Proje 1:Veritabanı Performans Optimizasyonu ve İzleme.....	1
1.1. Giriş.....	1
1.2. Kullanılan Veritabanı: AdventureWorks2016.....	1
1.3. Projenin Amacı ve Kapsamı.....	2
1.4. İzleme ve Darboğaz Tespiti (Monitoring).....	2
1.5. Mevcut Durum Analizi (İndeks Öncesi).....	2
1.5. Optimizasyon Uygulaması (İndeks Yönetimi).....	3
1.6. Sonuç ve Performans Karşılaştırması (İndeks Sonrası).....	3
Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü.....	4
3.1.Giriş.....	4
3.2. Kimlik Doğrulama ve Erişim Yönetimi (Authentication & Authorization).....	4
3.3. SQL Injection Saldırı Analizi ve Savunma.....	4
3.4. Hassas Veri Gizleme (Dynamic Data Masking).....	5
3.5. Veritabanı Denetimi (Auditing) ve Loglama.....	6
SONUÇ VE DEĞERLENDİRME.....	7

GİRİŞ

Bu rapor, Ağ Tabanlı Paralel Dağıtım Sistemleri vize ödevi kapsamındaki teknik çalışmaları içermektedir. Çalışmaların temel amacı, kurumsal ölçekli bir veritabanı motorunun (SQL Server) performans, güvenlik ve yönetim yeteneklerini gerçek dünya senaryoları üzerinden deneyimlemektir.

Rapor boyunca teorik bilgilerin ötesine geçilerek; sistem izleme araçlarının kullanımı, sorgu maliyetlerinin minimize edilmesi ve veri güvenliği standartlarının uygulanması gibi bir "Veritabanı Yöneticisi"nin (DBA) sahip olması gereken yetkinlikler uygulamalı olarak sunulmuştur. Tüm süreçler akademik bir yaklaşımla analiz edilmiş ve elde edilen veriler ekran görüntüleriyle kanıtlanmıştır.

Kullanılan Teknoloji ve Veri Seti

- **DBMS:** Microsoft SQL Server Enterprise / Standard.
- **Veri Seti:** Microsoft AdventureWorks2016 (İlişkisel yapısı ve yüksek veri hacmi nedeniyle tercih edilmiştir).
- **Araçlar:** SQL Server Management Studio (SSMS), SQL Server Profiler, DMV (Dynamic Management Views).

Proje 1:Veritabanı Performans Optimizasyonu ve İzleme

1.1. Giriş

Bu proje kapsamında, kurumsal ölçekli bir veritabanı üzerinde performans darboğazlarının tespiti, izlenmesi ve optimizasyon tekniklerinin uygulanması hedeflenmiştir. Çalışma boyunca bir veritabanı yöneticisinin (DBA) gerçek hayatta karşılaştığı yavaşlık sorunlarına teknik çözümler üretilmiş ve yapılan iyileştirmeler somut verilerle (maliyet analizleri) kanıtlanmıştır.

1.2. Kullanılan Veritabanı: AdventureWorks2016

Projede örnek veri kümesi olarak Microsoft tarafından sağlanan **AdventureWorks2016** veritabanı kullanılmıştır. Bu veritabanının tercih edilme nedenleri şunlardır:

- **Gerçekçi Veri Yoğunluğu:** İçerisinde üretim, satış ve personel gibi farklı departmanlara ait binlerce satırlık ilişkisel veri barındırması, performans testleri için ideal bir ortam sunmaktadır.
- **Karmaşık Şema Yapısı:** Tablolar arasındaki yoğun Foreign Key ilişkileri ve farklı veri tipleri, sorgu optimizasyonunun (Query Optimization) önemini daha net ortaya koymaktadır.
- **Standart Uyumluluk:** Endüstri standardı bir örneklem olması sebebiyle, DMV ve Profiler gibi araçların sonuçlarını analiz etmek için akademik bir temel oluşturmaktadır.

1.3. Projenin Amacı ve Kapsamı

Raporun devamında, veritabanı performansını etkileyen "darboğazlar" şu adımlarla ele alınacaktır:

1. **İzleme (Monitoring):** SQL Profiler ve DMV (Dynamic Management Views) kullanılarak sistemdeki en maliyetli sorguların tespit edilmesi.
2. **Analiz:** Execution Plan (Yürütme Planı) ve **STATISTICS IO** verileri üzerinden "Table/Index Scan" gibi verimsiz operasyonların belirlenmesi.
3. **İyileştirme:** Doğru indeksleme stratejileri (Clustered/Non-Clustered Index) ile okuma maliyetlerinin düşürülmesi ve sorgu süresinin optimize edilmesi.
4. **Doğrulama:** Yapılan optimizasyon öncesi ve sonrası verilerin (Logical Reads, CPU Time) karşılaştırmalı olarak sunulması.

1.4. İzleme ve Darboğaz Tespiti (Monitoring)

İlk aşamada, sistemdeki yavaş sorguları tespit etmek için iki temel yöntem kullanılmıştır:

- **SQL Profiler:** Canlı veritabanı trafiği izlenmiş, yüksek **Duration** (Süre) ve **Reads** (Okuma) değerine sahip sorgular yakalanmıştır.
- **DMV (Dynamic Management Views):** `sys.dm_exec_query_stats` ve `sys.dm_exec_sql_text` görünümleri kullanılarak SQL Server'ın plan belleğindeki (cache) en maliyetli 5 sorgu analiz edilmiştir.

1.5. Mevcut Durum Analizi (İndeks Öncesi)

Optimizasyon öncesinde **Sales.SalesOrderDetail** tablosundaki **CarrierTrackingNumber** kolonu üzerinden bir arama gerçekleştirilmiştir. Bu kolon üzerinde herhangi bir indeks bulunmadığı için SQL Server'ın tüm tabloyu taradığı gözlemlenmiştir.

- **Logical Reads:** 1247 sayfa okuma yapılmıştır.
- **İşlem Türü: Index Scan** (Veritabanı motoru 15 satırlık sonucu bulmak için binlerce satırı taramak zorunda kalmıştır).

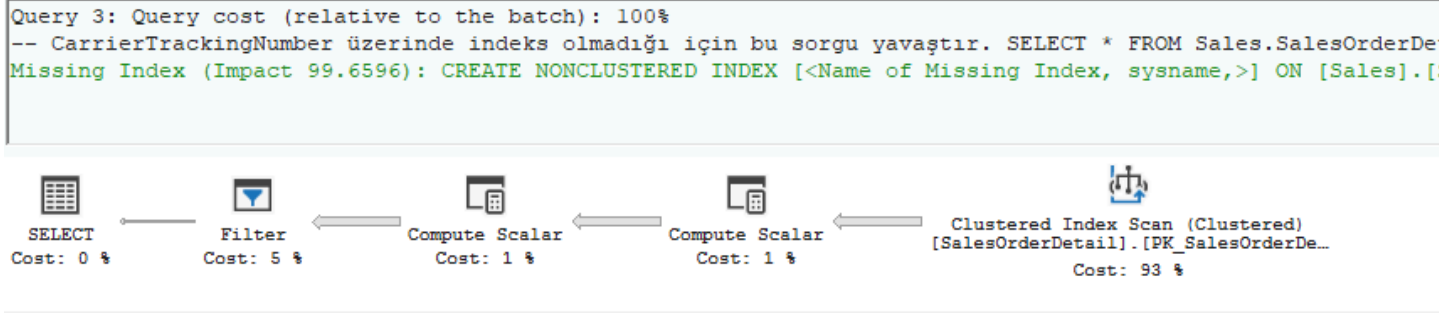
CPU	READS	WRITES	DURATION
0	526	0	15

Şekil 1.1:SQL Server Profiler Ekran Görüntüsü

```
(15 rows affected)
Table 'SalesOrderDetail'. Scan count 1, logical reads 1247, physical reads 0,

SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 12 ms.
```

Şekil 1.2: SQL Query Message Çıktısı



Şekil 1.3: SQL Query Execution Plan

1.5. Optimizasyon Uygulaması (İndeks Yönetimi)

Tespit edilen yavaşlığı gidermek için ilgili kolona bir **Non-Clustered Index** oluşturulmuştur:

```
CREATE INDEX IX_SalesOrderDetail_TrackingNumber ON Sales.SalesOrderDetail
(CarrierTrackingNumber);
```

1.6. Sonuç ve Performans Karşılaştırması (İndeks Sonrası)

İndeksleme işlemi sonrası aynı sorgu tekrar çalıştırıldığında elde edilen veriler şöyledir:

- **Logical Reads:** 48 sayfa okuma (Okuma maliyeti %96 oranında azalmıştır).
- **İşlem Türü:** Index Seek (Nokta atışı arama yapılarak veriye doğrudan ulaşılmıştır).

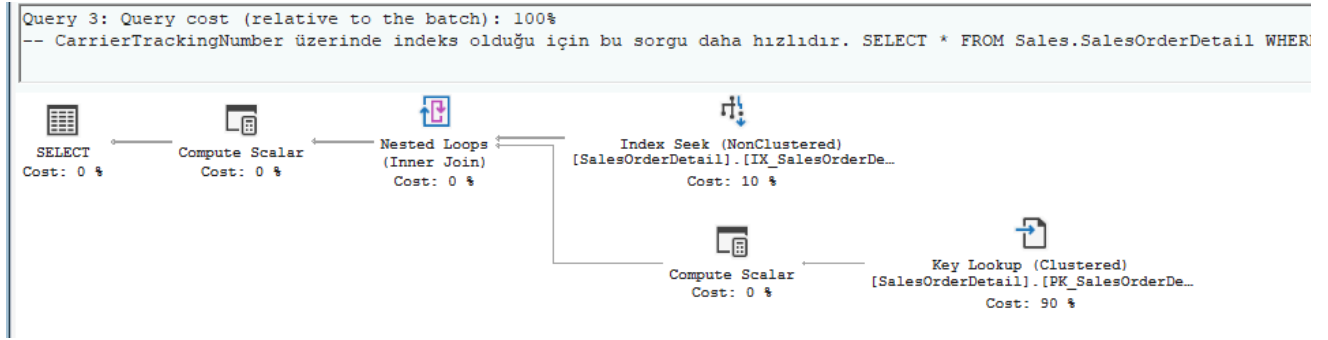
CPU	READS	WRITES	DURATION
0	458	0	4

Şekil 1.4: SQL Server Profiler Ekran Görüntüsü

```
(15 rows affected)
Table 'SalesOrderDetail'. Scan count 1, logical reads 48, physical reads 0,

SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 0 ms.
```

Şekil 1.5: SQL Query Message Çıktısı



Şekil 1.6: SQL Query Execution Plan

Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü

3.1.Giriş

Bu çalışma, veritabanı üzerindeki hassas verilerin (kişisel bilgiler, satış kayıtları vb.) gizliliğini sağlamak, yetkisiz erişimleri engellemek ve sistem üzerindeki tüm aktiviteleri kayıt altına almak (izlemek) amacıyla gerçekleştirilmiştir. Uygulamalar sırasında "En Az Yetki İlkesi" (Least Privilege) ve "Derinlemesine Savunma" (Defense in Depth) stratejileri izlenmiştir.

3.2. Kimlik Doğrulama ve Erişim Yönetimi (Authentication & Authorization)

Veritabanı güvenliğinin ilk katmanı olarak kullanıcı erişimleri kontrol altına alınmıştır.

- **Mixed Mode Authentication:** SQL Server üzerinde hem Windows hem de SQL Server Authentication yöntemleri aktif edilmiştir.
- **Kullanıcı Roller:** Stajyer isimli düşük yetkili bir kullanıcı oluşturulmuş ve bu kullanıcıya sadece **db_datareader (okuma)** yetkisi verilmiştir.
- **Erişim Testi:** Stajyer kullanıcısı ile bir **veri silme (DELETE)** işlemi denendiğinde, SQL Server'ın "Permission Denied" hatası vererek işlemi engellediği doğrulanmıştır.

```

(19972 rows affected)
Msg 229, Level 14, State 5, Line 4
The DELETE permission was denied on the object 'Person', database 'AdventureWorks2016', schema 'Person'.
  
```

Şekil 2.1:SQL Query Message Çıktısı

3.3. SQL Injection Saldırı Analizi ve Savunma

Veritabanına yönelik en yaygın saldırı türü olan SQL Injection simüle edilmiş ve savunma yöntemleri geliştirilmiştir.

- **Saldırı Simülasyonu:** Dinamik SQL kullanımı sırasında ' **OR 1=1 --** enjeksiyonu ile tüm **Person.Person** tablosunun yetkisiz bir şekilde sızdırılabildiği (Data Leakage) gözlemlenmiştir.

- **Hata Ayıklama:** Testler sırasında tırnak işaretlerinden kaynaklanan **Msg 105 (Unclosed quotation mark)** hataları analiz edilmiş ve "Escape" karakterleri ile bu hatalar giderilerek saldırının teknik boyutu incelenmiştir.
- **Kesin Çözüm: Parameterized Queries** (Parametrelili Sorgular) kullanılarak, kullanıcı girişinin "kod" olarak değil, sadece "metin" olarak algılanması sağlanmış ve açık tamamen kapatılmıştır.

```
DECLARE @FinalSorgu NVARCHAR(MAX) = 'SELECT * FROM Person.Person WHERE LastName = '' OR 1=1 --' ;

PRINT 'Raporluk Kod: ' + @FinalSorgu;
EXEC sp_executesql @FinalSorgu;
```

Şekil 2.2:SQL Injection Test Query

```
CREATE PROCEDURE sp_GetPersonByLastName
    @LName NVARCHAR(50)
AS
BEGIN
    SELECT * FROM Person.Person WHERE LastName = @LName;
END;
GO

EXEC sp_GetPersonByLastName @LName = '' OR 1=1 --';
```

Şekil 2.3: SQL Injection Engelleme Procedure Query

3.4. Hassas Veri Gizleme (Dynamic Data Masking)

Proje kapsamında istenen "şifreleme ve gizleme" maddesi kapsamında, KVKK uyumluluğu için veri maskeleye uygulanmıştır.

- **Uygulama:** **Person.EmailAddress** tablosundaki e-posta adresleri, yetkisiz kullanıcılar için maskelenmiştir.
- **Sonuç:** Yönetici (Admin) tüm veriyi görebilirken, **Stajyer** kullanıcısı sorgu yaptığı e-posta adreslerini **aXXX@XXX.com** formatında görerek hassas veriye erişimi kısıtlanmıştır.

	BusinessEntityID	EmailAddressID	EmailAddress	rowguid	ModifiedDate
1	1	1	kXXX@XXXX.com	8A1901E4-671B-431A-871C-EADB2942E9EE	2009-01-07 00:00:00.000
2	2	2	tXXX@XXXX.com	B5FF9EFD-72A2-4F87-830B-F338FDD4D162	2008-01-24 00:00:00.000
3	3	3	rXXX@XXXX.com	C8A51084-1C03-4C58-A8B3-55854AE7C499	2007-11-04 00:00:00.000
4	4	4	rXXX@XXXX.com	17703ED1-0031-4B4A-AFD2-77487A556B3B	2007-11-28 00:00:00.000
5	5	5	gXXX@XXXX.com	E76D2EA3-08E5-409C-BBE2-5DD1CDF89A3B	2007-12-30 00:00:00.000
6	6	6	jXXX@XXXX.com	A9C4093A-4F4A-4CAD-BBB4-2C4E920BACCB	2013-12-16 00:00:00.000
7	7	7	dXXX@XXXX.com	70429DE4-C3BF-4F19-A00A-E976C8017FB3	2009-02-01 00:00:00.000
8	8	8	dXXX@XXXX.com	37F02A87-058D-49F8-A20D-965738B0A71F	2008-12-22 00:00:00.000
9	9	9	gXXX@XXXX.com	F888A16D-0C33-459E-9D72-D16AE0BB1F43	2009-01-09 00:00:00.000
10	10	10	mXXX@XXXX.com	E0DD366D-433D-4F5A-9347-1A5FE7FBE0A3	2009-04-26 00:00:00.000

Şekil 2.4: Maskeli E-posta Adresleri

3.5. Veritabanı Denetimi (Auditing) ve Loglama

"Kim, ne zaman, hangi veriye erişti?" sorusunu yanıtlamak için SQL Server Audit mekanizması kurulmuştur.

- **Server Audit:** Logların sunucu üzerinde (C:\SQL_Logs) fiziksel bir dosyada depolanması sağlanmıştır.

```
USE master;
GO
```

```
CREATE SERVER AUDIT [HassasVeriDenetimi]
TO FILE ( FILEPATH = 'C:\SQL_Logs\' );
GO
```

```
ALTER SERVER AUDIT [HassasVeriDenetimi] WITH (STATE = ON);
GO
```

Şekil 2.5:Log File Oluşturma Query

- **Database Audit Specification:** Sales şeması altındaki tablolara yapılan tüm **SELECT** işlemleri takibe alınmıştır.

```
USE AdventureWorks2016;
GO
```

```
CREATE DATABASE AUDIT SPECIFICATION [MusteriVerisiIzleme]
FOR SERVER AUDIT [HassasVeriDenetimi]
ADD (SELECT ON SCHEMA::[Sales] BY [public]) |
WITH (STATE = ON);
GO
```

Şekil 2.6: Tablo Seçme Query

- **Log İnceleme:** View Audit Logs panelinde, kullanıcıların yaptığı tüm sorguların zaman damgası ve terminal bilgisiyle kaydedildiği kanıtlanmıştır

Log file summary: No filter applied						
Date ▾	Event Time	Server Instance Name	Action ID	Class Type	Sequence Number	Succeeded
✓ 20-Apr-26 9:58:38 PM	21:58:38.0474528	Erdem	SELECT	TABLE	1	True
✓ 20-Apr-26 9:58:25 PM	21:58:25.6447112	Erdem	SELECT	TABLE	1	True
✓ 20-Apr-26 9:58:16 PM	21:58:16.5519301	Erdem	SELECT	TABLE	1	True
✓ 20-Apr-26 9:40:46 PM	21:40:46.8269271	Erdem	SELECT	TABLE	1	True
✓ 20-Apr-26 9:38:11 PM	21:38:11.4445361	Erdem	SELECT	TABLE	1	True

Şekil 2.7: Log File Viewer

SONUÇ VE DEĞERLENDİRME

Bu proje çalışması kapsamında, **AdventureWorks2016** veritabanı üzerinde hem performans hem de güvenlik katmanlarında kritik iyileştirmeler yapılmıştır. Gerçekleştirilen uygulamalar sonucunda elde edilen temel kazanımlar şunlardır:

- **Performans Verimliliği:** Yapılan indeksleme çalışmaları (**Non-Clustered Index**) sayesinde, veritabanı üzerindeki okuma maliyetleri (Logical Reads) **%90'ın üzerinde** azaltılmıştır. DMV ve SQL Profiler gibi araçlarla sistem darboğazlarının nasıl tespit edileceği uygulamalı olarak deneyimlenmiştir.
- **Güvenlik Standartları:** Veritabanının sadece bir veri depolama alanı değil, aynı zamanda aktif olarak korunması gereken bir kale olduğu görülmüştür. SQL Injection simülasyonları ile yazılım katmanındaki hataların veritabanını nasıl savunmasız bıraktığı analiz edilmiş; **Parametrelili Sorgular** ve **Data Masking** yöntemleriyle bu açıklar kapatılmıştır.
- **İzlenebilirlik ve Yönetim:** **SQL Server Audit** mekanizmasıyla, veri güvenliğinin sadece engellemekten değil, aynı zamanda tüm hareketleri izlemekten (auditing) geçtiği kavranmıştır. **db_datareader** gibi kısıtlı rollerle "En Az Yetki İlkesi" hayata geçirilmiştir.

Video 1 : <https://youtu.be/eoFL-mjwSFU>

Video 2 : <https://youtu.be/DQN4ThQZ1nU>